Instituto Superior de Engenharia
Universidade Politécnica de Coimbra

Introdução às Redes de Comunicação

Licenciatura em Engenharia Informática

Docentes

- **Frederico Miguel Santos**
  - Teórica + P2 + P4
  - fred@isec.pt
- **José Fernando Rosado**
  - P3 + P5 + P6
  - jfr@isec.pt
- **Celina Jorge Baptista**
  - P1 + P7
  - celina.batista@isec.pt

Avaliação

Componente teórica

- Exame escrito nas épocas de avaliação, de acordo com o mapa de exames
- Admitidos a exame APENAS:
  - Alunos com mínimos nos testes laboratoriais
  - Mínimo de participação em 50% das aulas de laboratório
- Valoração: 12 valores
- Mínimos : 40% de 12 valores

Componente laboratorial

- 2 testes laboratoriais – datas propostas e sujeitas a confirmação
  - 14 de outubro
  - 18 de novembro
- Valoração: 8 valores (4 valores cada um)
- Mínimos : 35% de 8 valores
- Não é possível melhorar a avaliação dos testes laboratoriais

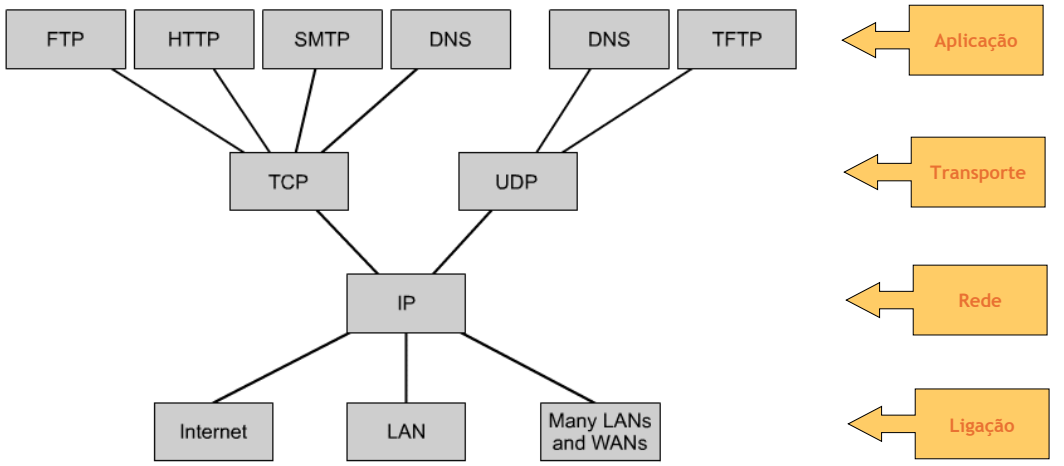
Instituto Superior de Engenharia
Universidade Politécnica de Coimbra

Arquitetura TCP/IP

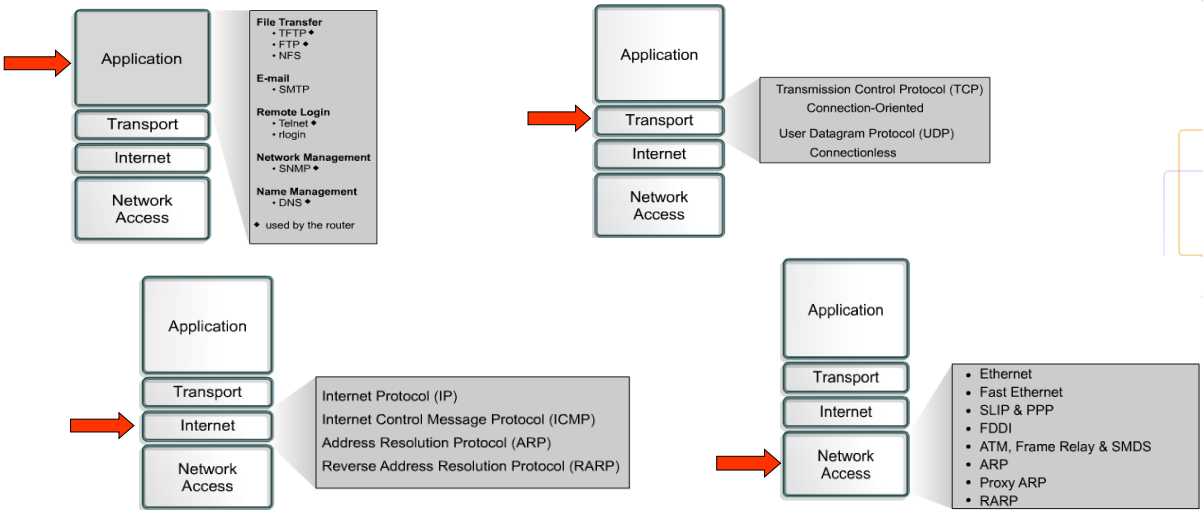
Introdução às Redes de Comunicação

Licenciatura em Engenharia Informática

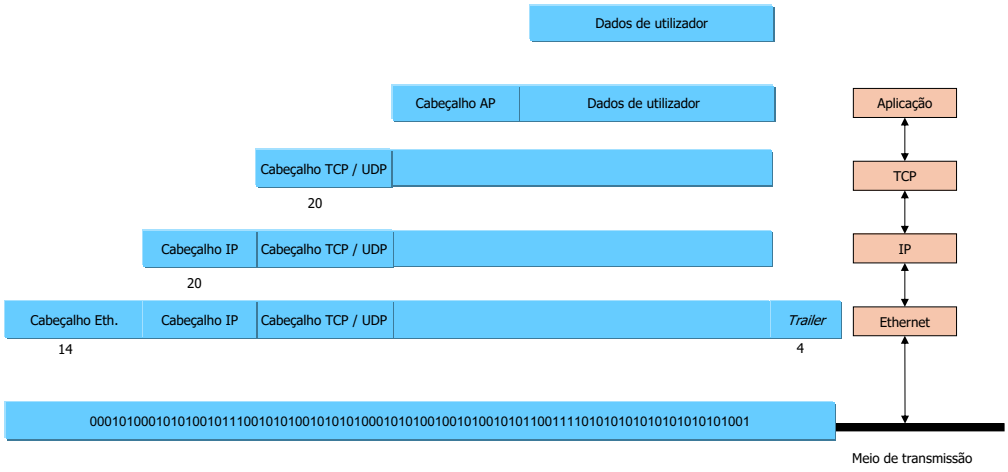
Pilha protocolar



Pilha protocolar



Pilha protocolar

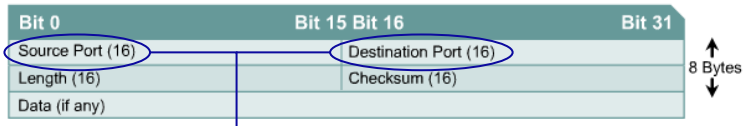


Protocolo UDP

- **UDP – User Datagram Protocol**
- Do tipo não orientado a ligação
- Não é fiável
- Preferível ao TCP quando não é exigida uma fiabilidade de 100%, devido à sua reduzida sobrecarga protocolar
- Exemplos
  - TFTP e DNS recorrem ao UDP e realizam os seus próprios controlos de erros
  - Transmissão de voz ou de imagens em que erros ocasionais são aceitáveis

Protocolo UDP

- Cabeçalho de um datagrama UDP

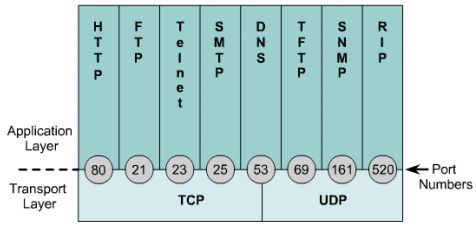


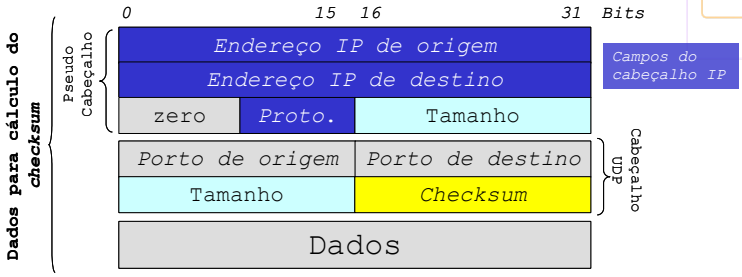
- **Portos**
  - A localização de um processo na Internet é dada por **3 coordenadas**:
    - **Endereço IP**
    - **Porto**
    - **Protocolo de transporte**

Exemplos: <129.12.34.5, 53, UDP> e <129.12.34.5, 53, TCP>

  - Comando `netstat` (e.g., `netstat -an`)

Protocolo UDP





Instituto Superior
de Engenharia
Universidade Politécnica de Coimbra

Sockets UDP

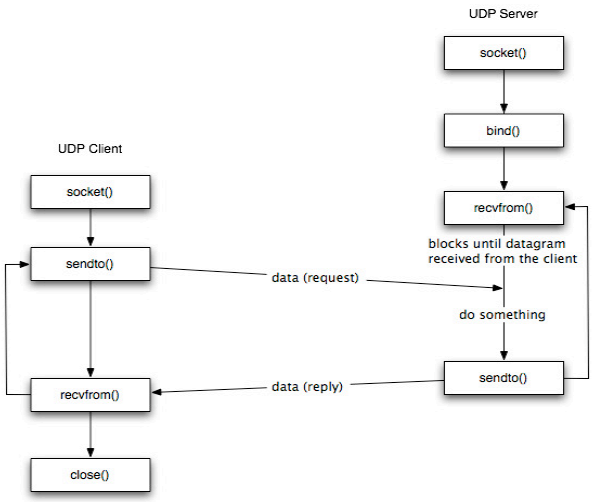
Introdução às Redes de Comunicação

Licenciatura em Engenharia Informática

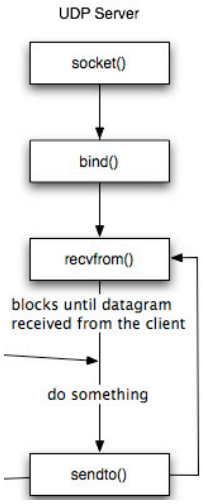
Sockets

- Permite a troca de mensagens entre os processos de uma máquina/aplicação servidor e de uma máquina/aplicação cliente
- Tipos de Sockets:
  - STREAM (TCP) fornecem garantia na entrega
  - DATAGRAM (UDP) são mais simples e sem garantia
- Conexões definidas por porto (0 - 65535)
- Servidor pode suportar múltiplas conexões simultâneas (mais de 1 cliente na mesma porta)
  - Simples de implementar em DATAGRAM (UDP)

Fluxo de sockets UDP

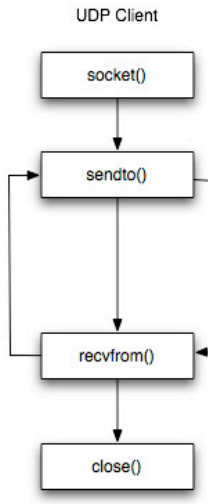


Implementação – servidor UDP



- **socket()**
  - Define o tipo de *socket*: datagram (UDP)
- **bind()**
  - Especifica o endereço (IP e porto) por onde serão esperadas ligações
- **recvfrom() / sendto()**
  - Realiza a troca de mensagens

Implementação – cliente UDP



- **socket()**
  - Define o tipo de *socket*: datagram (UDP)
- **sendto() / recvfrom()**
  - Realiza a troca de mensagens
- close() → em Winsock é a função **closesocket()**
  - Fecha o socket

Sockets em Windows (Winsock)

- API **Winsock2** (`ws2_32.lib`), que segue a especificação dos *sockets* de Berkeley, mas exige inicialização e libertação explícitas da biblioteca
  - Funções Windows Socket API. Ex: `WSAStartup()` e `WSACleanup()`
- Implementação:
  - Cabeçalho: Incluir `<winsock2.h>` e `<ws2tcpip.h>`
    evitar usar `<winsock.h>` pois é a versão antiga do Winsock
  - Ligação (Linking): Ligar a biblioteca `ws2_32.lib`
    MS Visual Studio usa-se `#pragma comment(lib, "ws2_32.lib")`
    MinGW ou GCC usa-se `-lws2_32` na linha de comandos
  - Utilização da API:
    - `WSAStartup()` antes da criação do *socket*
    - `WSACleanup()` no final ou em situações de erro que levem ao encerramento
    - `WSAGetLastError()` permite obter o código do último erro

Sockets em Windows (Winsock)

```
int WSAStartup(WORD wVersionRequired, LPWSADATA lpWSAData);
```

- **`wVersionRequired`**
  - A versão mais alta do Winsock que a aplicação pretende utilizar
    - O byte de menor peso especifica a versão maior (*major*)
    - O byte de maior peso especifica a versão menor (*minor*)
    - Cria-se tipicamente com a macro `MAKEWORD(major, minor)`
      - Ex: `MAKEWORD(2, 2) // usa apenas a versão 2 da API`

- **`lpWSAData`**
  - Apontador para uma estrutura do tipo `WSADATA` onde a função escreve os detalhes da implementação suportada pelo SO

Sockets em Windows (Winsock)

```
SOCKET socket(int af, int type, int protocol);
```

- **`af`** (Address Family)
  - Família de endereços (ou domínio) da comunicação. Define o formato dos endereços de rede que serão usados:
    - `AF_INET`: Endereçamento IPv4 (32 bits)
    - `AF_INET6`: Endereçamento IPv6 (128 bits)
    - `AF_BTH`: Bluetooth (suportado nativamente no Winsock)
- **`type`** (Socket Type)
  - Protocolo da transmissão dos dados:
    - `SOCK_STREAM`: Fornece fluxos de bytes bidirecionais, fiáveis e baseados em ligação (orientado à ligação). Garante entrega por ordem, sem duplicação e sem limites de mensagens. Usado com TCP
    - `SOCK_DGRAM`: Fornece datagramas sem ligação (*connectionless*), não fiáveis e de comprimento fixo máximo. Não garante ordem nem entrega. Usado com UDP
    - `SOCK_RAW`: Acesso direto ao nível do protocolo IP (permite manipular ou inspecionar cabeçalhos de rede; no Windows exige privilégios de administrador)
- **`protocol`** (Protocol)
  - Protocolo de transporte concreto. Na grande maioria dos casos passa-se 0, permitindo que o SO escolha o protocolo por omissão para o par (af, type). Pode tomar os valores:
    - `IPPROTO_TCP`: Para ligações TCP (af = AF_INET/AF_INET6, type = SOCK_STREAM)
    - `IPPROTO_UDP`: Para datagramas UDP (af = AF_INET/AF_INET6, type = SOCK_DGRAM)
    - `IPPROTO_RAW`: Para *sockets* em modo bruto

Sockets em Windows (Winsock)

```
int bind(SOCKET s, const struct sockaddr *name, int namelen);
```

- **s**
  - Descritor do *socket* retornado por `socket()`
- **name**
  - Apontador genérico para a estrutura com os detalhes de rede
    - Contém a informação do **IP** e do **porto**
  - Na prática, instancia-se uma estrutura específica para a família pretendida:
    `struct sockaddr_in` para IPv4
    `struct sockaddr_in6` para IPv6
- **namelen**
  - Dimensão em *bytes* da estrutura de endereço passada
    - ex: `sizeof(struct sockaddr_in)`

Sockets em Windows (Winsock)

```
struct sockaddr_in {
    short           sin_family;      // Familia de endereços (AF_INET)
    u_short         sin_port;        // Número do porto (em Network Byte Order)
    struct in_addr  sin_addr;        // Endereço IP de 32 bits (em Network Byte Order)
    char            sin_zero[8]; }; // Bytes de enchimento para alinhar a 16 bytes
```

- **sin_family**
  - Especifica a família de endereços. Deve conter o valor `AF_INET`
- **sin_port**
  - O porto de comunicação TCP ou UDP (de 0 a 65535)
  - Tem de ser obrigatoriamente convertido *Network Byte Order* (*Big-Endian*) usando a macro `htons()` (*host to network short*)
- **sin_addr**
  - O endereço IPv4. Também tem de estar em *Network Byte Order*. Pode ser configurado através de:
    - `htonl(INADDR_ANY)`: No caso de um servidor que deva escutar em todas as interfaces locais (0.0.0.0)
    - `inet_pton(AF_INET, "192.168.1.10", &addr.sin_addr)`: Abordagem recomendada para converter uma cadeia de carateres com notação decimal pontuada para o formato binário
      - `addr.sin_addr.s_addr = inet_addr("192.168.1.10")`: A evitar, não verifica se o IP está correto!

Sockets em Windows (Winsock)

```
int sendto(SOCKET s, const char *buf, int len, int flags,
    const struct sockaddr *to, int tolen);

int recvfrom(SOCKET s, char *buf, int len, int flags,
    struct sockaddr *from, int *fromlen);
```

- **buf** (buffer de dados)
  - Apontador para a zona de memória contendo os *bytes* da mensagem
- **len** (comprimento dos dados)
  - Número de *bytes* de `buf`
- **flags** (modificadores de comportamento)
  - Ajusta a forma como o envio é processado pela pilha de rede. Na maioria dos casos passa-se 0
- **to** ou **from** (endereço de destino ou de origem)
  - Na prática, usa-se a estrutura específica `sockaddr_in` para IPv4
- **tolen** ou **fromlen** (tamanho do endereço)
  - Dimensão em *bytes* da estrutura de endereço. Ex: `sizeof(struct sockaddr_in)`

Sockets em Windows (Winsock)

- Configuração opcional de tempo máximo de espera na receção

```
int setsockopt(SOCKET s, int level, int optname, char *optval, int optlen);
```

- **s**
  - Descritor do *socket* retornado por `socket()`
- **level**
  - Nível da opção a definer (usar: `SOL_SOCKET`)
- **optname**
  - Nome da opção (usar: `SO_RCVTIMEO`)
- **optval**
  - Apontador para a zona de memória com os parâmetros da opção
    - O tempo máximo de espera é representado por um inteiro sem sinal (`DWORD`) expresso em mili-segundos (ms)
- **optlen**
  - Tamanho em *bytes* de `optval`
    - Tamanho do inteiro (`DWORD`)